

Explain the full AERIS software architecture to me like I am the system owner who wants deep understanding, not just code changes.

I want a rigorous explanation of:

1. what the software currently is,
2. why it is structured this way,
3. how the pieces connect,
4. why modularity matters here specifically,
5. what is already solved,
6. what is not solved yet,
7. and how future phases logically extend from the current architecture.

Use the following project state as ground truth.

PROJECT PURPOSE

AERIS is being built as a modular aircraft design platform for:

- deterministic geometry generation
- modular sampling
- dataset generation
- future aerodynamic analysis
- future ML-oriented data products
- future traceable engineering workflows

This is not meant to remain a pile of scripts. It is being turned into a structured platform with interfaces, registries, pipelines, manifests, and CLI control. The architecture guide already frames the project as a modular factory where interfaces define machine classes, implementations plug into registries, and pipelines orchestrate execution while keeping components decoupled.

CURRENT SOFTWARE STRUCTURE

Explain clearly the role of each main area:

- src/aeris/commands/
- src/aeris/common/
- src/aeris/geometry/
- src/aeris/generators/
- src/aeris/dataset/
- src/aeris/pipeline/
- src/aeris/aero/

Explain what each one is responsible for and how they interact.

CURRENT IMPLEMENTED ARCHITECTURE

A. CLI layer

There is a Typer CLI with command groups for geometry, dataset, pipeline, and version.

Explain why the CLI is important and why production execution should go through it instead of through scattered scripts. :contentReference[oaicite:15]
{index=15}

B. Geometry layer

There is now a formal geometry generator contract and registry.

Active generator:

- bwb_segmented_v1

Explain:

- what a generator base class is
- why generator resolution happens through a registry
- why typed config construction belongs inside the generator
- why this is better than hardcoded script logic

C. Determinism model

Explain the deterministic rule now enforced:

- sampler seed controls sampling
- once a design sample exists, geometry is a pure deterministic function of that sample

Explain why this matters for:

- reproducibility
- dataset trust
- ML
- future certification-oriented traceability

The BWB planform logic explicitly documents the move away from downstream RNG contamination, so use that as part of the explanation. :contentReference[oaicite:16]{index=16}

D. Sampling layer

There is a dataset sampler base class and registry.

Validated samplers:

- lhs_v1
- random_v1

Explain:

- why sampler modularity matters
- why removing lhs_seed in favor of sampler_seed was correct
- why seeding belongs to the sampler abstraction, not specifically to LHS

E. Dataset pipeline

Explain the real orchestration path:

raw config -> typed config -> sampler resolution -> explicit samples -> generator.run_full_case -> artifacts + metadata + failures + manifest

Explain:

- why dataset generation should not reimplement geometry
- why artifacts and manifests are important
- why metadata/failure tables matter
- why no-plot mode matters for production batch runs

F. AeroSandbox path

Explain that the system now supports optional AeroSandbox geometry build and what that means:

- it does NOT mean aero solving exists yet
- it means a geometry representation useful for later aero work can already be constructed and stored in metadata fields like aspect ratio / xsecs count
- this is an important bridge toward the aero phase

G. Smoke / validation / hardening

Explain the meaning of:

- bad-config failure handling
- invalid sampler failure handling
- missing config CLI rejection
- 500-case no-plot stress test
- 20-case AeroSandbox-enabled validation

Explain why these are not just tests, but proofs that the architecture is actually working.

LEGACY SCRIPT CONNECTION

I also want you to explain how the older reference/prototype scripts relate to the AERIS architecture.

These scripts show future needs:

- CST geometry and exact section extraction
- geometry reconstruction/export to AeroSandbox / OpenVSP / RCAIDE

- AVL aero sweeps with raw outputs and strip data
- scalar feature extraction
- SDF / graph dataset building
- later GNN consumption

Explain:

- what these scripts reveal about the real long-term pipeline
- why they justify modularity
- why they should be treated as migration references instead of copied directly
- which architectural seams they imply:
 1. geometry generators
 2. geometry views/export adapters
 3. aero solvers
 4. dataset builders
 5. later extraction policies

Use the fact that the old AVL workflow and full dataset builder scripts already show solver- and builder-specific logic that clearly belongs behind future interfaces.

LIMITATIONS / HONEST GAPS

Explain clearly what is still limited today:

- only one true geometry family exists
- sampler implementations are still family-specific in practice
- geometry views/export adapters are not formal plugin families yet

- dataset builders are not formal plugin families yet
- aero architecture is only a shell and must become real next

NEXT LOGICAL EXTENSION

Then explain analytically why the next phase should be the aero architecture, and what that means in software terms:

- solver base class
- solver registry
- aero input models
- aero result models
- failure states
- provenance and logging
- dataset integration
- lightweight preparation for future geometry-view/export seams

I want the explanation to feel like a textbook chapter for this specific project:

- not generic software theory
- not shallow
- not vague
- not hand-wavy

Explain the current AERIS architecture as it truly exists now, why it is correct, where it is still incomplete, and how it should evolve next without collapsing back into script chaos.